Project report : Road segmentation

Bennani Ghalia, Berruyer Mylène, Hajoub Mehdi

***Abstract*—This project explores the challenging task of road segmentation using satellite imagery, a key component in modern applications such as autonomous driving, urban planning, and traffic analysis. Our objective is to accurately distinguish roads from their surrounding environment in satellite images. Leveraging advanced machine learning techniques, we aim to develop a model that can effectively identify road networks with high precision. This report details our methodology, from data preprocessing and model selection to the application of metrics for performance evaluation. We provide insights into the strengths and weaknesses of different segmentation models, highlighting our approach's significance in contributing to the broader field of image-based geographical analysis.**

I. Introduction

This project aims to accurately segment roads in a collection of images, targeting high precision and F1-scores. We utilized a dataset of 100 satellite or aerial images sourced from Google Maps, accompanied by corresponding ground-truth images where pixels are labeled as either 'road' or 'background'. Our approach involved multiple phases, starting with the evaluation of various machine learning models to establish baseline performances. Subsequently, we focused on refining the most promising model, optimizing its performance through iterative hyperparameter tuning and exploring a range of data augmentation techniques.

II. Data preprocessing

Before initiating the analysis, it was critical to preprocess our data to ensure its quality and compatibility with the model.

A. Missing data

Before proceeding, it is crucial to ensure the integrity of our data. This involves checking the completeness of each image, ensuring that no part is missing or that the images are not incomplete. This manual verification is essential to guarantee the quality of the data before any further processing.

B. Preprocessing masks

The images in our dataset were already largely consistent in format and color characteristics. As for the masks, the primary preprocessing step involved verifying that they contained only binary values (0s and 1s). Originally encoded in RGB, these masks had values ranging from 0 to 237, predominantly zeroes. To address this, we converted them to gray scale and than applied a filter to convert any pixel with a value greater than 0 to 1. This transformation ensured that our masks accurately represented binary information, and it also helped balance our dataset by increasing the number of 'road' pixels.

III. Metrics

Now that our data set is ready, we need to know which of the models performs the best on it. Our aim is to maximize the accuracy and the f1 score of our prediction while minimizing the metrics.

Intersection over Union (IoU) and Dice coefficient are chosen for model performance evaluation due to their effectiveness in quantifying the accuracy of segmentation tasks. IoU measures the overlap between predicted and actual segments, providing a clear metric for boundary precision. The Dice coefficient, emphasizing overlap size, is sensitive to the balance of false positives and negatives, making it ideal for detailed segmentation assessment. Both metrics are robust against class imbalance, crucial in tasks like road segmentation.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}} = \frac{|A \cap B|}{|A \cup B|}$$

The IoU based loss is equal to 1 - IoU and therefore measures the portion of the prediction that does not overlap with its ground truth, we need to minimize this quantity.

Moreover, the dice loss = 1 - Dice Coefficient (DSC) , where the dice coefficient is the ratio between 2 times the area of overlap between the prediction and the ground truth and the total area of the concatenation of both items. We also want to minimize it.

$$\text{DSC} = \frac{2|A \cap B|}{|A| + |B|}$$

To evaluate the models, we adopted a strategy that involved testing them on previously unseen data, specifically, the training set was excluded. We partitioned our dataset into three distinct segments for each execution: 80% for training, 10% for validation, and the remaining 10% for testing. Post-training, we deployed the models on their respective testing sets and compared them based on various metrics: IoU scores, Dice similarity coefficient (DSC) scores, F1 scores, and a visual assessment of their predictions for qualitative analysis.

IV. Models

A. U-net

The U-Net architecture's effectiveness for semantic segmentation arises from its unique structure that is specifically tailored to capture both context and fine-grained details, which are essential in differentiating between the object classes and background at the pixel level. Here's why the combination of its architecture components makes it a good candidate for semantic segmentation. We implemented our code from this github

Encoding (Contracting Path): The encoder is a series of convolutional and max pooling layers that progressively downsample the image, reducing its spatial dimensions while increasing the depth of feature maps. This encoding process captures the context and abstract features of the image, which are essential for understanding the overall structure and for distinguishing between different objects and the background in semantic segmentation.

Decoding (Expansive Path): The decoder progressively upsamples the feature maps and recovers spatial details and resolution. It uses transposed convolutions or up-convolutions to perform this upsampling, which is crucial for reconstructing a detailed segmentation map that aligns closely with the original image size.

Skip Connections: The skip connections are perhaps the most distinctive feature of U-Net. They bridge the gap between the encoding and decoding paths by combining the high-resolution features from the encoder with the upsampled features in the decoder. These connections directly concatenate feature maps from the encoder to the corresponding decoder layers, providing precise localization information that would otherwise be lost due to downsampling. This is particularly important for semantic segmentation, where the exact boundaries of objects are critical.

Final Segmentation Map: The architecture is symmetrical, and the output from the final layer of the decoder is a segmentation map that categorizes each pixel in the image. The combination of context, detailed localization, and fine-grained features provided by the encoder-decoder paths, enhanced by skip connections, allows the network to generate highly accurate segmentation results.

The design of the U-Net architecture, particularly its use of skip connections that reintroduce localization information lost during downsampling, coupled with the encoding and decoding paths, makes it exceptionally well-suited for tasks that require detailed pixel-level predictions, such as semantic segmentation. It effectively combines the understanding of the global context of the image (captured in the downsampling process) with the ability to precisely delineate the shape and location of objects (reconstructed during the upsampling process).

Finally, a 1x1 convolution layer is used to map each 64-feature vector to the desired number of categories. In total, this network comprises 23 feature-processing layers.

The repeated application of 3x3 convolutional filters allows the network to learn spatial hierarchies of features. At the initial layers, the filters detect simple patterns, such as edges and corners. As we progress deeper into the network, subsequent layers combine these initial features to recognize more complex structures.

The convolution operation in deep learning can be defined as follows:

$$(S * K)(i,j) = \sum_m \sum_n S(i-m, j-n) \cdot K(m,n) \qquad (1)$$

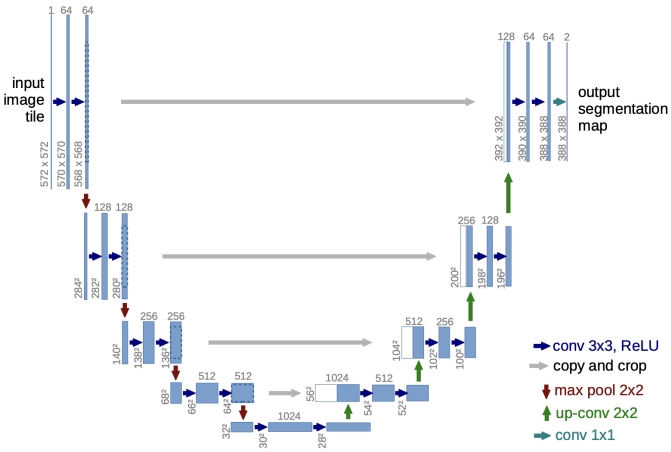


Fig. 1: "U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations." [2]

Where:

- S is the input image (signal),
- K is the kernel (filter),
- $*$ denotes the convolution operation,
- (i, j) are the coordinates in the output feature map,
- $\sum$ denotes the summation,
- m and n are the indices that traverse the filter.

This formula represents the sum of element-wise products between the filter and the input signal at each position of the filter over the input. Convolutional layers extract and learn spatial hierarchies of features, ReLU introduces necessary non-linearities enabling the model to learn complex patterns, and max pooling reduces dimensionality, enhancing feature robustness and computational efficiency. Together, they enable U-Net to accurately capture both the context and detailed features necessary for precise pixel-level segmentation.

B. SAM

We aimed to assess the SAM (Segment Anything Model), our second targeted model for testing. SAM comprises three key elements: an image encoder, a flexible prompt encoder, and a fast mask decoder.[1] We implemented our code from this github

The image encoder, driven by considerations of scalability and robust pre-training methodologies, employs a sparsely adapted Vision Transformer (ViT) architecture pre-trained with MAE, a pre-training technique where parts of the input are masked, and the model learns to predict them. This technique is used to develop a robust understanding of image features., specifically designed for processing high-resolution inputs. The image encoder performs a single pass per image and can be applied prior to prompting the model.

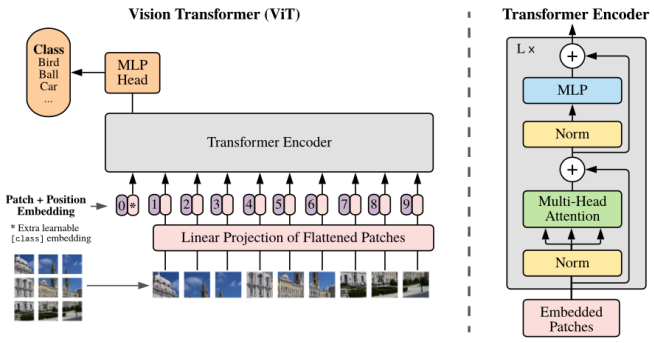


Fig. 2: Vision Transformer ViT Architecture.

Regarding the prompt encoder, two prompt sets are taken into account: sparse (points, boxes, text) and dense (masks). Points and boxes are represented by positional encodings combined with learned embeddings for each prompt type, while free-form text is processed using a pre-existing text encoder from CLIP. Dense prompts, such as masks, undergo embedding through convolutions and are element-wise summed with the image embedding.

The mask decoder efficiently maps the image embedding, prompt embeddings, and an output token to a mask. This design incorporates a modified Transformer decoder block, succeeded by a dynamic mask prediction head. Our adapted decoder block utilizes prompt self-attention and cross-attention in two directions (prompt-to-image embedding and vice versa) to update all embeddings. Following the execution of two blocks, we up-sample the image embedding, and an MLP (Multi-Layer Perceptron) maps the output token to a dynamic linear classifier. This classifier then calculates the mask foreground probability at each image location.

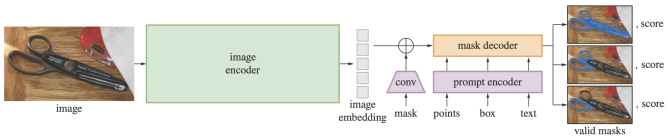


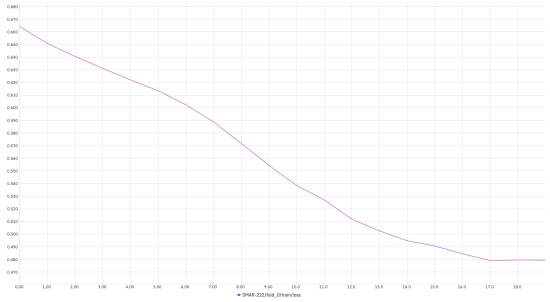
Fig. 3: Segment Anything Model (SAM) overview. A heavyweight image encoder outputs an image embedding that can then be efficiently queried by a variety of input prompts to produce object masks at amortized real-time speed. For ambiguous prompts corresponding to more than one object, SAM can output multiple valid masks and associated confidence scores.

V. Results

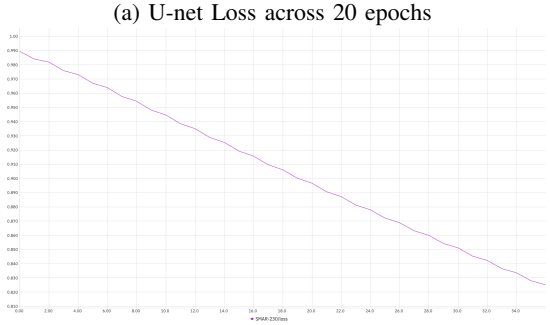
For fair comparison we decided to run a first training with the equivalent hyperparameters (learning rate = 1e-4, optimizer = Adam , number of epochs = 20, batch size = maximum batch size possible with model loaded on the GPU's memory), for both U-Net and SAM. We chose the Adam optimizer because as seen in the lectures it adjusts learning rates individually for each parameter, allowing for efficient optimization and convergence, especially in complex loss landscapes. We than monitored the loss thanks to the neptune tool which allows us to keep track on the loss as the model trains. Once the training was over, we test the two models on more than 10 different subsets and obtained the following results which allowed us to compare them.

Model	IOU	Dice
SAM	0.55 (0.022)	0.75(0.021)
UNET	0.22 (0.023)	0.66 (0.0168)

TABLE I: Comparison of IOU, Dice, and F1 scores for SAM and UNET

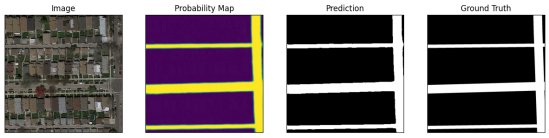


(a) U-net Loss across 20 epochs

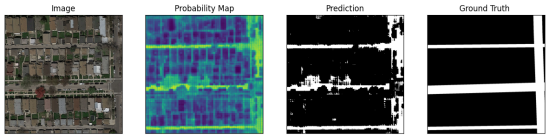


(b) SAM loss across 20 epochs

Fig. 4: SAM loss vs U-net Loss with learning rate = 1e-4, optimizer = Adam , number of epochs = 20, batch size = maximum batch size possible with model loaded on the GPU's memory

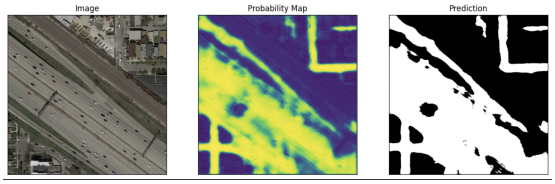


(a) U-net prediction straight roads

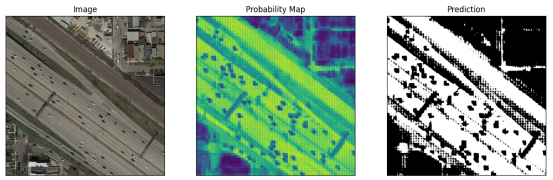


(b) SAM prediction straight lines

Fig. 5: SAM vs Unet predictions for straight roads



(a) U-net prediction inclined roads



(b) SAM prediction inclined lines

Fig. 6: SAM vs Unet prediction for inclined roads

At first glance, the U-Net outperforms the SAM model on our two evaluation metrics. We will examine the curves depicting the cost function's evolution as training progresses as we can see on figure 4. These curves exhibit a very similar trend, including minimal oscillations. This allows us to conclude that both models started on an equal footing in terms of the hyperparameters used, making their comparison feasible and coherent for analysis.

A more in-depth analysis of the predictions generated by each model (see Figure 5 and 6 - for the predictions of each model) reveals distinct differences. U-Net provides more precise road detection with less visual noise, while SAM appears to exhibit more unwanted variations.

It is worth nothing, however, that SAM performs well in detecting diagonal roads, a capability where U-Net shows certain limitations, particularly for roads that are neither strictly vertical nor horizontal.

Despite more significant errors and a tendency towards noise in some predictions, SAM manages, in certain cases, to identify vehicles and even traffic signs—an aspect that sparks particular interest.

As a result of our analyses, we have decided to continue our study by focusing our efforts on the U-Net model, owing to its overall superior performance in road detection, even though SAM demonstrates remarkable specific capabilities in certain contexts.

VI. U-net optimization

A. Data augmentation

Previously, we identified a limitation of the U-net in segmenting inclined roads, which isn't surprising given the predominance of straight, perpendicular roads in our dataset and only a few images featuring inclined roads. Consequently, it was a logical step to introduce a specific transformation for the already trained U-net – a random rotation of 45°, assigned a higher probability of occurrence compared to other transformations. These modifications are applied randomly and not systematically, ensuring the integrity of the data is maintained and avoiding the introduction of bias or undue influence on the model's learning process. By initiating with a model that was trained on the original dataset and subsequently introducing it to an augmented dataset (featuring partially transformed images), we effectively influence not just the diversity of the images but also, in a way, increase the volume of data itself.

B. Methodology

Initially, we were training repeatldy a new U-net network on different hyperparameters settings, with or without data augmentation (transforms) hoping to find across trials the setting that would make the difference. Eventually we face the conclusion that the first hyperparameters that we used, which gave us early good results (F1 = 0.839 , Accuracy = 0.913) on AIcrowd, were the best, and, the small variations of hyperparameters didn't have the drastic impact that we wanted to have on our model. This is when we thought of transfer learning.

Our initial model, post its first training session, lacked exposure to augmented data and struggled significantly with inclined roads. This observation inspired us to use the existing model as a foundation for further training. We maintained the same hyperparameters but introduced data augmentation this time. The goal was to refine a model already proficient to a variety of geometrical and optical set-ups.

We executed this process iteratively, selecting the top-performing model from the training on the dataset with random image transformations and evaluating its effectiveness on our test set, followed by AIcrowd submission. If the model achieved a higher score on AIcrowd, it was designated as the new benchmark, forming the basis for subsequent training sessions. Notably, the score fluctuations between trainings were quite significant, a reflection of the randomness inherent in the image transformations.

Between each training iteration, we alternated the learning rate, switching between 1e-5 and 1e-4, starting by the 1e-4.

Transfer learning, when applied recursively on a model while alternating between high and low learning rates, can yield both potential benefits and risks.

C. Results

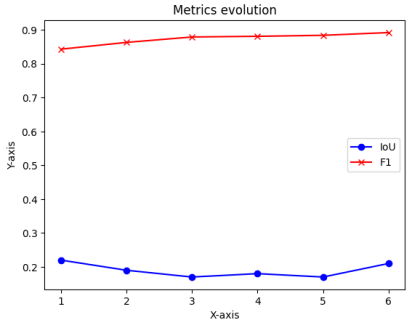


Fig. 7: Iterations of the transfer learning algorithm switching between learning rates 1e-5 and 1e-4.

Surprisingly enough this recursive allowed us to improve our scores and metrics along successful trials. A learning rate of 1e-4 seems to be a better value for our model.

VII. Ethical risks

Ethics in road segmentation refers to the consideration of moral principles and values when developing, implementing, and using road segmentation technologies. Here are some ethical considerations in road segmentation:

Bias and Fairness: Machine learning models used for road segmentation may inherit biases present in the training data. If the training data is not diverse and representative, the model may perform poorly in certain conditions or environments. This can result in unfair treatment of different demographic groups or regions. Privacy: Road segmentation technologies often rely on various sensors, including cameras, which can capture images of the surrounding environment. Privacy concerns arise when these technologies inadvertently capture sensitive information, such as faces or license plates, and may be used inappropriately. Environmental Impact: The development and deployment of road segmentation technologies may have environmental consequences, such as increased energy consumption or the need for additional hardware. Evaluating and mitigating the environmental impact of these technologies is an ethical consideration.

Now, let's focus on the bias and fairness part. Indeed, individuals and communities in regions with underrepresented or biased training data are the one impacted by this risk. The negative impact consists of unfair treatment, discrimination, and inaccuracies in road segmentation for specific demographic groups or regions, potentially leading to safety hazards or unequal access to transportation services. The likelihood of unfairness in road segmentation can vary based on several factors including the availability and representativeness of training data, the demographic diversity of the population, and the distribution of resources. Certain regions or populations may be more susceptible to unfairness in road segmentation. Here are some considerations: Low-Income Areas, Rural and Remote Areas, Developing Countries, Areas with Limited Technological Adoption ...

Unfortunately, we did not take the risk into account in our project. Indeed, we could have expanded the dataset to include more diverse and representative samples, actively addressing underrepresented communities or regions, and implemented adversarial training techniques to address bias during model training ensuring equal performance across different demographics and regions. However, obtaining diverse and representative data i challenging, especially in cases where certain demographic groups or regions were historically underrepresented.

VIII. Conclusion

Finally, the results of our model on the test set after the data pre-processing and training are: 89.2% of f1-score and the secondary score of 94,1 %. There is indeed room for improvement. For example, we could test other models, cross validate over different optimizers and take into account the ethical part in our project. Furthermore, the ranges used for hyper-parameters selection could be narrowed so we could test more precise values... Despite that, we are satisfied with our results and we keep in mind the things that we could have done to enhance it.

References

[1] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, et al. Segment anything. *arXiv preprint arXiv:2304.02643*, 2023.

[2] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. pages 234–241, 2015.